

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

*I am **R.Monika(192324281)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Monika

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

$$(a|b)^*a(a|b)|(a|b)$$

2. a) Illustrate the predictive parser for the following grammar. 4

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of

$$S \rightarrow SS^+ | SS^* | a$$

String: "aa+a" 4

4. Eliminate left recursion from the grammar $S \rightarrow Sa|Sb|c$ 2
5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$$A \rightarrow abcD | abcE | f$$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

$$Degree - F = Degree - C * 1.8 + 32$$

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

```
sum = 0;
```

```
for (i = 1; i < 5; i++)
```

```
    sum = sum + i;
```

Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.

$$S \rightarrow aABb, A \rightarrow c|\epsilon, B \rightarrow d|\epsilon$$

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table.

not. Show the parser moves for the input string **acdb**.

9. Write a simple Lex program to recognize and count identifiers in a given input.2

10. Write the rule to Eliminate Left Recursion and Left Factoring.2

11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3

12. Consider the following grammar:

$$S \rightarrow pBB \rightarrow qB|r$$

Find the leftmost derivation and construct the parse tree for the input string pqqrr.3

13. Consider Grammar: $E \rightarrow E+T | T$ $T \rightarrow T * F | F$ $F \rightarrow (E) | id$

Eliminate left recursion and find FIRST and FOLLOW 5

14. For the following grammar, determine whether the string **abcc** is accepted using shift-reduce parsing. Show all shift and reduce operations.

10

$$S \rightarrow ABA \rightarrow aA|aB \rightarrow cB|c$$

15. For the following grammar, verify whether the string **abb~~c~~** is valid using shift-reduce parsing.10

$$S \rightarrow aABA \rightarrow bA | bB \rightarrow c$$

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar:10

$$S \rightarrow SS \rightarrow aABA \rightarrow bA | cB \rightarrow d$$

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow iEtSS | a, S \rightarrow eS | \epsilon, E \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

$$A \rightarrow aAB | aA | a$$

$$B \rightarrow bB | b$$

19. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow ABCA \rightarrow a | \epsilon B \rightarrow r | \epsilon C \rightarrow b | \epsilon$$

20. Construct the LL(1) Parsing Table for the following grammars.7

$$S \rightarrow 1S\# | qABCA \rightarrow a | bbDB \rightarrow a | \epsilon C \rightarrow b | \epsilon D \rightarrow c | \epsilon$$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

R. MONIKA

192324281

CSA1402 -

COMPILER DESIGN

04/08/2026

CO2 - AT2 - SHORT ANSWER

①. Regular Expression: $(a|b)^* a(a|b)(a|b)$

Ans:-

$a|b$ (or) $A+b$

$\rightarrow (a) \rightarrow (b)$

76
100



②. Predictive Parsing:

$S \rightarrow (L) | a$

$L \rightarrow L, S | S$

$S \rightarrow (L)$

$S \rightarrow a$

$L \rightarrow L/S$

$L' \rightarrow SL'$

$L \rightarrow S/S$



$$\text{FIRST}(S) = \{ (, a \}$$

$$\text{FIRST}(L) = \{ (, a \}$$

$$\text{FIRST}(L') = \{ (, a, \$ \}$$

$$\text{FOLLOW}(S) = \{ \$ \}$$

$$\text{FOLLOW}(L) = \{), \$ \}$$

$$\text{FOLLOW}(L') = \{), \$ \}$$

	a	(	)	\$
S	$S \rightarrow (L) a$	$S \rightarrow (L) a$		
L				
L'			$L' \rightarrow SL'$	

③.

$$S \rightarrow SS+ | SS* | a$$

Given string : "aa+a"

Predictive Parser

$$S \rightarrow SS+$$

$$S \rightarrow SS*$$

$$S \rightarrow a$$

$$S \rightarrow SS S' | a$$

$$S' \rightarrow + | *$$

Stack	Input	Output
\$S	aa+a\$	$S \rightarrow SS+$
\$ S SS	aa+a\$	$S \rightarrow a$
\$+aa	aa+a\$	
\$+	+a\$	
\$	a\$	

④ Left recursion: $S \rightarrow Sa | Sb | c$

$$S \rightarrow Sa$$

$$S \rightarrow Sb$$

$$S \rightarrow c$$

$$S \rightarrow ass' | bss' | c$$

⑤ Predictive Parsing:

$$A \rightarrow abcD | abCE | f$$

$$A \rightarrow abc A' | f$$

$$A' \rightarrow D | E$$

} Left factoring

⑥ Phase of compiler:

$$\text{Degree}_F = \text{Degree}_c * 1.8 + 32$$

$$id_1 = id_2 * 1.8 + 32$$

① Lexical Analyzer:

$$id_1 = \text{Degree}_F$$

identifier

$$id_2 = \text{Degree}_c$$

identifier

*

Multiplication Operator

1.8

constant (INT)

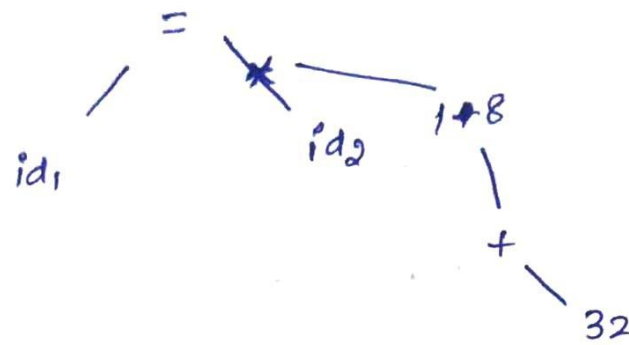
+

Addition operator

32

constant (INT)

②. Syntax Analyzer:



③. Semantic Analyzer:

$id_1 = id_2 * 1.8 + 32$

④. Intermediate code Generator:

temp1 = id₂
temp2 = temp1 * 1.8

temp3 = 32

temp4 = temp2 + temp3

temp5 = temp4

⑤. Code Optimizer

temp1 = id₂ * 1.8

temp2 = 32

temp3 = temp1 + temp2

temp4 = temp3.

⑥. Code Generator:

MOVF ~~r1~~, id₂, r₂

MULF * 1.8, r₂

ADDF 32, r₂

MOVF r₂, r₁

④.

C code fragment:

sum = sum + i ;

①. Lexical Analyzer

Token

Operator

sum

identifier

=

Equal operator

sum

identifier

+

addition operator

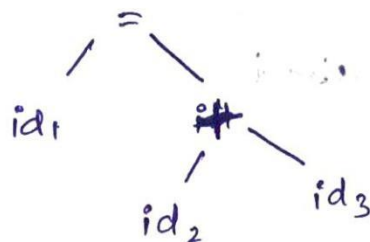
i

identifier

;

Separator

②. Syntax Analyzer:



③. Semantic Analyzer:

id₁ = id₂ + id₃

④. Intermediate code Generator

temp1 = id₂

temp2 = id₃

temp3 = temp1 + temp2

temp4 = temp3

⑤. code optimizer

temp1 = id₂ + id₃

temp2 = temp1

⑥. code Generator

MOVF id₂, r₂

ADDF id₃, r₂

MOVF r₂, r₁

8.

$S \rightarrow aABb$

$A \rightarrow c | \epsilon$

$B \rightarrow d | \epsilon$

$FIRST(S) = \{a\}$

$FIRST(A) = \{c, \epsilon\}$

$FIRST(B) = \{d, \epsilon\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{\$ \}$

$FOLLOW(B) = \{\$, b\}$

Non-Terminal	a	b	c	d	\$
S	$S \rightarrow aABb$				
A			$A \rightarrow c$		$A \rightarrow \epsilon$
B		$B \rightarrow \epsilon$		$B \rightarrow d$	$B \rightarrow \epsilon$

Given Input String: abcd

+

Stack	Input	Output
\$	abcd \$	
\$bBAa	abcd \$	$S \rightarrow aABb$
\$bBA	bcd \$	
\$BA	cd \$	$A \rightarrow c$
\$BC	cd \$	
\$B	d \$	$B \rightarrow d$
\$	\$	

1). Lex Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int x = a + b * c; int count = 0;
```

```
    if (x = a - z || A - z)
```

```
        print ("identifier");
```

```
        count++;
```

```
    else {
```

```
        print ("not identifier");
```

```
    return;
```

2) user / desktop / flex ex: no: 1.1

lex gcc yy.c

a.exe

output: 3

(10)

Rule of left Recursion Elimination

* If the same terminal present on both side then we need to do

Left Recursion

Ex:- $E \rightarrow E + T / T$

Rules to Left factoring

* If it has common prefixes then we need to do Left factoring.

Ex:-

$S \rightarrow ist s / ist s cs / a$

$S \rightarrow ist s s' / a$

$s' \rightarrow \epsilon / cs$

(11) Actions Performed by shift-reduce Parser

> shifting

> Reducing

> Accepting

Example: Given string: $id + id * id$

Stack	Input Buffer	Output	
\$	$id + id * id \$$		$E \rightarrow E + T / T$
$\$ id$	$+ id * id \$$	shift	$T \rightarrow T * F / F$
$\$ F$	$+ id * id \$$	$F \rightarrow id$	$F \rightarrow (E) / id$
$\$ +$	$id * id \$$	shift	$E \rightarrow FE'$
$\$ F +$	$id * id \$$		$E' \rightarrow + TE' / \epsilon$
			$T \rightarrow FT'$
			$T' \rightarrow * FT' / \epsilon$
			$F \rightarrow (E)$
			$F \rightarrow id$

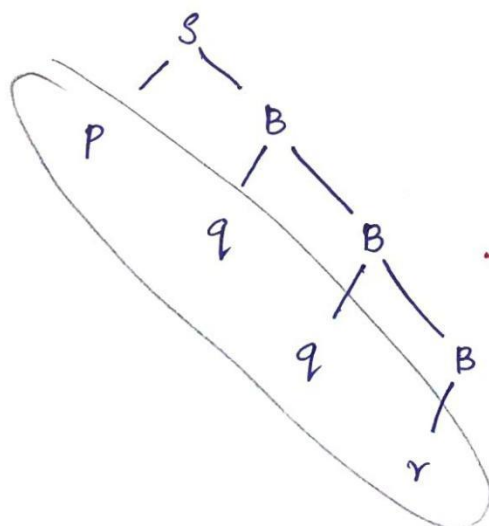
12.

$$S \rightarrow pB$$

$$B \rightarrow qB/r$$

Leftmost derivation:

Given string: $ppqr$



Given Grammar:

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

Left recursion:

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

$$E \rightarrow E + T$$

$$T \rightarrow T * F$$

$$F \rightarrow (E)$$

$$E \rightarrow T$$

$$T \rightarrow F$$

$$F \rightarrow id$$

$$E \rightarrow TE'$$

$$T \rightarrow FT'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T' \rightarrow *FT' / \epsilon$$

FIRST

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \epsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

FOLLOW

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$\text{FOLLOW}(T) = \{ \$,), + \}$$

$$\text{FOLLOW}(T') = \{ \$,), + \}$$

$$\text{FOLLOW}(F) = \{ \$,), *, + \}$$

(14). Shift reduce Parsing:

$$S \rightarrow AB$$

$$A \rightarrow aA / a$$

$$B \rightarrow CB / C$$

input string : abcc

Stack

\$

\$a

\$A

\$Ab

Input

abcc\$

bcc\$

bcc\$

cc\$

Output

shift

reduce $A \rightarrow a$

shift

shift

Stack	Input	Output
\$AbC	c\$	reduce $B \rightarrow c$
\$AbB	c\$	shift
\$AbBC	\$	reduce $B \rightarrow c$
\$AbBB	\$	Not accepted

The given string abcc is not accepted.

(15) Given grammar:

$$S \rightarrow aAB$$

$$A \rightarrow bA$$

$$A \rightarrow b$$

$$B \rightarrow c$$

Input string: abbc

SHIFT-REDUCE PARSING:-

Stack	Input Buffer	Output
\$	abbc \$	shift
\$a	bbc \$	shift
\$ab	bc \$	shift
\$abb	c \$	Reduce $A \rightarrow b$
\$aba	c \$	Reduce $A \rightarrow bA$
\$aa	c \$	shift
\$aac	\$	reduce $B \rightarrow c$
\$aab	\$	reduce $S \rightarrow aAB$
\$S	\$	Accepted

String abbc is Accepted

16.

Given Grammar:

$$S' \rightarrow S$$

$$S \rightarrow aAB$$

$$A \rightarrow bA \mid c$$

$$B \rightarrow d$$

- Canonical collection of LR(0) items.
- SLR(1) Parsing table

I₀:

$$S' \rightarrow \cdot S$$

$$S \rightarrow \cdot aAB$$

$$A \rightarrow \cdot bA$$

$$A \rightarrow \cdot c$$

$$B \rightarrow \cdot d$$

$$\text{Goto}(I_0, S) = I_1$$

$$S' \rightarrow \cdot S \quad \times$$

$$S \rightarrow \cdot aAB \Rightarrow S \rightarrow a \cdot AB$$

$$\text{Goto}(I_0, A) = I_2$$

$$A \rightarrow \cdot bA$$

$$A \rightarrow \cdot c$$

$$\text{Goto}(I_0, B) = I_3$$

$$B \rightarrow \cdot d$$

$$\text{Goto}(I_1, S) = I_4$$

$$S \rightarrow a \cdot AB$$

$$\text{Goto}(I_2, A) = I_5$$

$$A \rightarrow bA \cdot$$

$$\text{Goto}(I_4, B) = I_6$$

$$S \rightarrow aA \cdot B$$

$$\text{Goto}(I_6, B) = I_7$$

$$S \rightarrow aAB \cdot$$

FIRST:

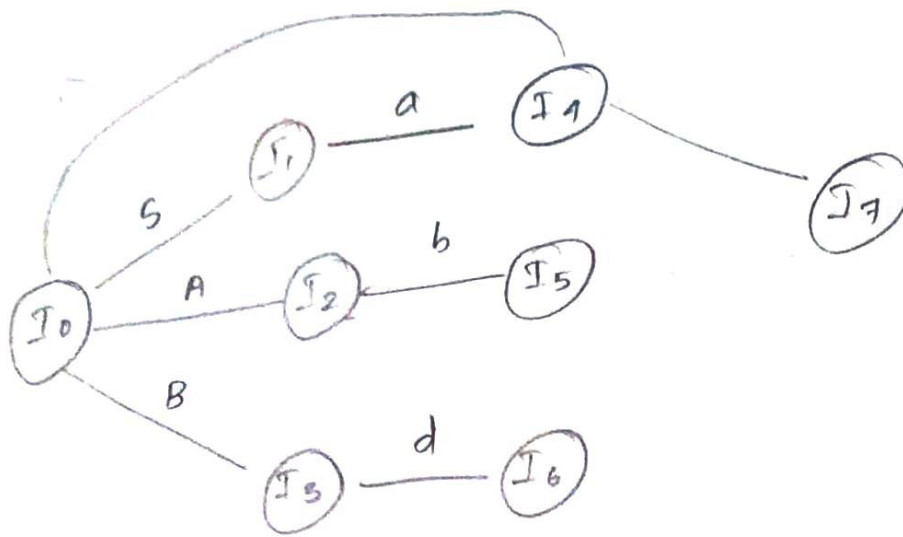
$$\begin{aligned} \text{FIRST}(S) &= \{a, \epsilon\} \\ \text{FIRST}(S') &= \{a, \epsilon\} \\ \text{FIRST}(A) &= \{b, c\} \end{aligned}$$

$$\text{FIRST}(B) = \{d\}$$

FOLLOW:-

$$\begin{aligned} \text{FOLLOW}(S) &= \{\$ \} \\ \text{FOLLOW}(S') &= \{\$ \} \\ \text{FOLLOW}(A) &= \{\$, b, c\} \end{aligned}$$

$$\text{FOLLOW}(B) = \{\$, a, b, c\}$$



State (I)	a	b	c	d	\$	S	A	B
0	S ₁				Accept	1	2	3
1			S ₂				5	
2								
3		r ₅						
4			r ₃			5	7	
5	r ₃							
6								
7								

(7)

$$S \rightarrow iEsS' / a$$

$$S' \rightarrow eS / \epsilon$$

$$E \rightarrow b$$

$$\text{FIRST}(S) = \{i, a\}$$

$$\text{FIRST}(S') = \{e, \epsilon\}$$

$$\text{FIRST}(E) = \{b\}$$

$$\text{FOLLOW}(S) = \{\$, e\}$$

$$\text{FOLLOW}(S') = \{\$, e\}$$

$$\text{FOLLOW}(E) = \{\$, e\}$$

Predictive Parsing Table.

Terminals

Non-terminals	a	b	e	i	\$
S	$S \rightarrow a$			$S \rightarrow ietss'$	
S'			$S' \rightarrow e$		$S' \rightarrow \epsilon$
E		$E \rightarrow b$			

18.

$$A \rightarrow aAB \mid aA \mid a$$

$$B \rightarrow bB \mid b.$$

Left factoring:-

$$A \rightarrow aAA' \mid a$$

$$A' \rightarrow B \mid A$$

$$B \rightarrow bB'$$

$$B' \rightarrow B \mid \epsilon$$

19.

LL(1) Parsing :

$$S \rightarrow ABC$$

$$A \rightarrow a \mid \epsilon$$

$$B \rightarrow r \mid \epsilon$$

$$C \rightarrow b \mid \epsilon$$

$$\text{FIRST}(S) = \{a, \epsilon\}$$

$$\text{FIRST}(A) = \{a, \epsilon\}$$

$$\text{FIRST}(B) = \{r, \epsilon\}$$

$$\text{FIRST}(C) = \{b, \epsilon\}$$

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{\$, r\}$$

$$\text{FOLLOW}(B) = \{\$, b, r\}$$

$$\text{FOLLOW}(C) = \{\$, b, r\}$$

20

LL(1) Parsing table

$S \rightarrow \epsilon \mid A \mid B \mid C$

$A \rightarrow a \mid b \mid D$

$B \rightarrow a \mid \epsilon$

$C \rightarrow b \mid \epsilon$

$D \rightarrow c \mid \epsilon$

$FIRST(S) = \{\epsilon, A, B, C\}$

$FIRST(A) = \{a, b\}$

$FIRST(B) = \{a, \epsilon\}$

$FIRST(C) = \{b, \epsilon\}$

$FIRST(D) = \{c, \epsilon\}$

$FOLLOW(S) = \{\$, \epsilon\}$

$FOLLOW(A) = \{\$, a, b\}$

$FOLLOW(B) = \{\$, a, \epsilon\}$

$FOLLOW(C) = \{\$, a, b, \epsilon\}$

$FOLLOW(D) = \{\$, a, b, \epsilon\}$

3